

UDESC

Departamento de Ciência da Computação

Estrutura de Dados 2

**Análise comparativa da complexidade algorítmica de
árvores AVL, rubro-negras e B**

Alunos: Gabriela Alves Ilezyszyn, Heitor Linhares Caetano e Pedro Henrique Dias
Professor: Allan Rodrigo Leite

Junho
2026

Conteúdo

1	Objetivo do Trabalho	1
2	Metodologia	2
2.1	Contagem de comparações	2
3	Implementação	3
3.1	Árvore AVL	3
3.2	Árvore Rubro-negra	3
3.3	Árvores B	4
4	Resultados	5
4.1	Análise dos resultados	5
4.2	Análise da Operação de Adição	5
4.3	Análise da Operação de Remoção	6
5	Considerações Finais	7

1 Objetivo do Trabalho

O objetivo do trabalho é analisar e comparar a complexidade computacional das operações de inserção e remoção de nós em árvores AVL, rubro-negras e B, considerando também o custo de balanceamento. Para isso, foram usados conjuntos de dados aleatórios com tamanhos variando de 1 a 10 mil. Os resultados foram apresentados em gráficos que relacionam o esforço temporal das operações, seguidos de uma discussão sobre o comportamento e a eficiência das diferentes estruturas apresentadas.

2 Metodologia

Para a criação dos conjuntos aleatórios, foi criado um vetor de tamanho 10 mil em que cada posição armazena o próprio índice. Após isso, é usado um [algoritmo de embaralhamento](#) chamado Fisher-Yates que randomiza o vetor com base em valores aleatórios obtidos com a função *rand*.

A partir desse vetor, o experimento foi construído de forma a avaliar o crescimento das estruturas de maneira *contínua*. Ou seja, as árvores não são destruídas e recriadas a cada ciclo, mas sim crescem um tamanho de passo (400) a cada iteração, chegando a 10.000 na última.

Em cada ponto de amostragem Quando o experimento chega em um dos pontos de amostragem (um múltiplo do tamanho do passo), o esforço computacional das operações é registrado. Para a inserção, é registrado apenas o custo das últimas 400 chaves (janela entre o passo analisado e o próximo). Isso faz com que o comportamento seja isolado e consigamos coletar apenas o custo da parte que importa, sem ser "diluído" pelo custo das inserções anteriores.

Já para a remoção, no mesmo ponto, primeiramente são criadas cópias temporárias das árvores que estamos criando. Após isso, os elementos delas são copiados para um vetor auxiliar, embaralhados novamente para que tenhamos uma ordem de remoção aleatória, e aí sim são removidos da estrutura temporária, com as comparações sendo contabilizadas. Isso faz com que possamos contabilizar o custo de remoção em determinado tamanho sem interromper o crescimento das árvores principais.

2.1 Contagem de comparações

Para estabelecer um método de comparação entre a complexidade das diferentes árvores, se faz necessário o uso da comparação como unidade de esforço computacional. Comparações são todas as partes do código-fonte que estabelecem uma mudança de fluxo condicional ao sistema. Na linguagem C, elas são invocadas em toda aparição das *keywords* `if`, `for`, `while`, `do{...}while`, além do operador ternário `?`.

Contudo, nem toda ocorrência desses termos é contabilizada para os fins deste trabalho. A justificativa para isso é de que nem toda comparação no código é inerente à estrutura analisada, sendo muitas vezes apenas parte da organização do fluxo do programa. O principal exemplo de comparação não contabilizada em nosso código são as variáveis e condições de laços de repetição, como `j > i`, operadores de verificação de ponteiros, como `if(raiz == NULL)` ou `if(!raiz)`, e validações de alocações de memória.

Essas operações são produtos apenas de decisões de implementação e gerenciamento de fluxo em linguagem C, e não representam o custo "matemático" do algoritmo. Portanto, foram contabilizados apenas os testes de relação entre chaves e de elementos armazenados, esse sim próprios a cada estrutura de dados.

3 Implementação

3.1 Árvore AVL

A árvore AVL foi implementada de maneira similar a demonstrada em aula, porém com uma pequena diferença: cada nó também armazena um inteiro que representa sua altura. Essa altura é atualizada a cada inserção ou remoção e é usada no cálculo do fator de balanceamento, que se torna muito mais simples com essa informação já em mãos, sendo apenas a diferença entre as alturas das subárvores esquerda e direita ($FB = altura(esquerda) - altura(direita)$). Caso o valor absoluto do fator de balanceamento atinja 2, as rotações simples ou duplas são acionadas para reestruturar a árvore.

Esse armazenamento das alturas de cada nó ajuda significativamente na eficiência da árvore, sendo o padrão na maioria das implementações disponíveis na *internet*. Se a estrutura precisasse percorrer recursivamente por todas suas subárvores para descobrir suas alturas a cada inserção ou remoção, a operação de balanceamento teria maior complexidade, sendo $O(n)$ no pior dos casos, diminuindo a vantagem de se usar uma árvore auto-balanceada.

Quando cada nó armazena explicitamente sua altura, o cálculo do fator de balanceamento se torna uma operação de esforço constante ($O(1)$).

3.2 Árvore Rubro-negra

A árvore Rubro-Negra foi implementada garantindo as propriedades fundamentais de balanceamento através da atribuição de cores (vermelho ou preto) a cada nó. Diferente da AVL, a Rubro-Negra assegura que o caminho mais longo da raiz até uma folha não seja maior que o dobro do caminho mais curto, mantendo a altura da árvore em $O(\log n)$.

A estrutura mantém a consistência da árvore através de regras rigorosas: Todo nó é ou vermelho ou preto, a raiz é sempre preta, e não podem existir dois nós vermelhos adjacentes. Sempre que uma inserção ou remoção viola essas propriedades, a árvore utiliza um conjunto de rotações e, crucialmente, a alteração das cores dos nós para restaurar o equilíbrio.

Esse armazenamento da cor em cada nó é o detalhe que confere a eficiência desta estrutura. Enquanto a AVL prioriza uma busca mais rápida, a Rubro-Negra é otimizada para cenários de alta rotatividade de dados. Como o rebalanceamento é feito através da mudança das cores e rotações são menos frequentes que na AVL, a árvore tende a apresentar um desempenho superior em sistemas onde inserções e remoções são tão frequentes quanto buscas.

3.3 Árvores B

A implementação da Árvore B foi projetada para ser flexível em relação ao seu parâmetro de ordem, que define a capacidade de ocupação de cada nó. Diferente das árvores AVL e Rubro-Negra, que são estruturalmente binárias, a Árvore B agrupa múltiplas chaves e ponteiros em um único bloco de memória. Na implementação, a capacidade máxima de um nó foi definida como $2 \times \text{ordem}$ chaves.

Para otimizar o desempenho computacional, a busca pela posição correta de uma chave dentro do vetor de um nó não é feita de forma sequencial, mas sim através de uma *busca binária* (implementada na função `pesquisa_binaria`). Embora árvores B de maior ordem (como a ordem 10) possuam menos níveis (reduzindo a descida na árvore), elas acomodam vetores maiores em cada nó, o que eleva a quantidade de comparações feitas pela busca binária local.

A operação de adição ocorre com a localização da folha adequada e a inserção ordenada da nova chave. Caso a quantidade de chaves do nó ultrapasse o limite máximo estabelecido, é disparada a condição de transbordo (*overflow*). Nesse cenário, o algoritmo realiza a operação de *split* (divisão): o nó é particionado em dois e a chave mediana é promovida para o nó pai. Esse processo pode ocorrer em cascata, subindo pela estrutura e, no limite, dividindo a raiz e aumentando a altura da árvore.

Por fim, a operação de remoção foi realizada de forma que, para evitar lacunas no meio da árvore, caso a exclusão seja solicitada em um nó interno, a chave é previamente substituída por seu sucessor imediato (que sempre reside em um nó folha). A remoção física ocorre exclusivamente nas folhas. Após a exclusão, caso o número de chaves do nó caia abaixo do mínimo aceitável (condição de *underflow*, que equivale à ordem da árvore), o algoritmo tenta realizar o empréstimo de uma chave de um de seus nós irmãos adjacentes através do pai. Se ambos os irmãos estiverem no limite mínimo, é realizada a operação de fusão (*merge*) do nó com seu irmão e a chave separadora do pai, um processo que diminui o tamanho do nó pai e pode propagar o *underflow* até a raiz da árvore.

4 Resultados

4.1 Análise dos resultados

Nesta seção, analisamos o esforço computacional, medido através do número de comparações, das operações de adição e remoção para as estruturas Árvore AVL, Árvore Rubro-Negra e Árvore B (nas ordens 1, 5 e 10).

4.2 Análise da Operação de Adição

Conforme ilustrado na Figura 1, a Árvore Rubro-Negra apresentou o desempenho mais eficiente. Este resultado decorre do seu critério de balanceamento menos rígido em comparação à AVL. Enquanto a AVL realiza rotações frequentes para garantir que a diferença de altura entre subárvores seja mínima, a Rubro-Negra aceita um desbalanceamento maior, o que reduz o custo de manutenção da estrutura.

Sobre a Árvore B, observa-se uma relação inversa entre a ordem e o número de comparações: ordens menores exigiram menos comparações do que ordens maiores. Embora ordens maiores reduzam a altura da árvore, o custo de realizar a busca binária dentro de um nó com um vetor de chaves extenso supera a economia obtida pela redução do número de nós visitados.

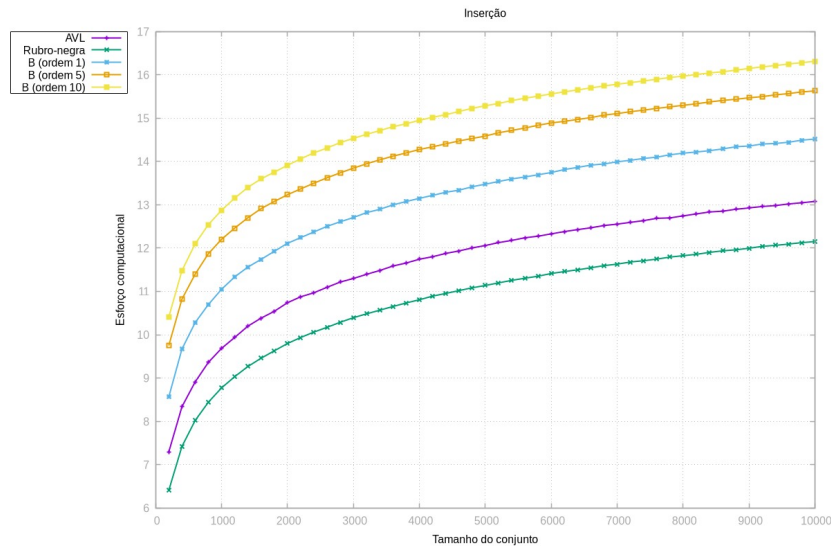


Figura 1: Comparação do número de comparações na operação de adição.

4.3 Análise da Operação de Remoção

Conforme ilustrado na figura 2, a operação de remoção, sendo a mais complexa, evidenciou o custo estrutural de cada árvore. A Árvore B de ordem 1 apresentou o maior custo de processamento. Como o nó possui uma capacidade reduzida, a condição de *underflow* é atingida com frequência, forçando o algoritmo a realizar diversas operações de fusão e empréstimo de chaves. A Árvore Rubro-Negra, novamente, mostrou-se a estrutura mais equilibrada para cenários de alta rotatividade, mantendo um custo previsível e otimizado.

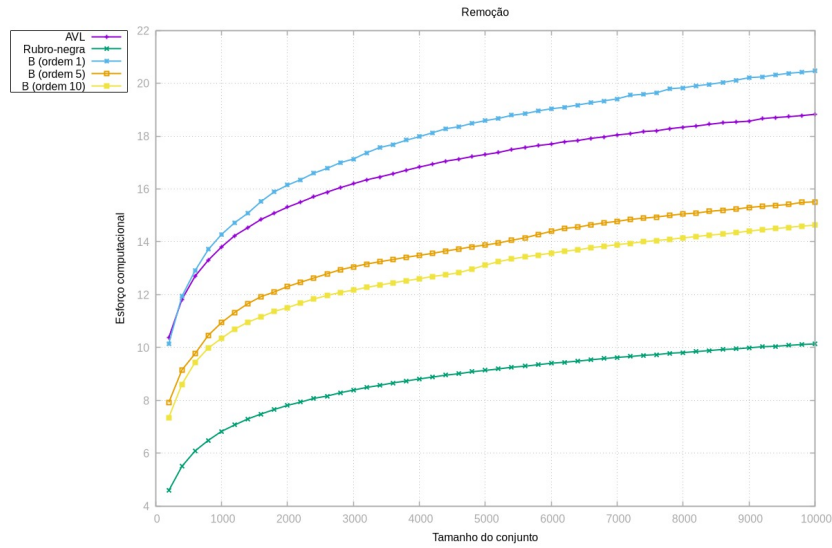


Figura 2: Comparação do número de comparações na operação de remoção.

5 Considerações Finais

O estudo demonstrou que a eficiência de uma estrutura de dados não depende apenas da sua complexidade assintótica teórica ($O(\log n)$), mas da sua implementação prática. Enquanto a AVL é eficiente para buscas, a Rubro-Negra demonstrou superioridade prática em inserções e remoções devido à redução de custos de balanceamento.

Para a Árvore B, conclui-se que o parâmetro de ordem atua como um ajuste fino (*fine-tuning*): ordens muito pequenas elevam o custo de manutenção estrutural (*underflow*), enquanto ordens muito grandes elevam o custo de processamento local (busca binária em vetores longos). O desempenho ideal, portanto, depende do perfil de acesso da aplicação, sendo que estruturas com rebalanceamento flexível tendem a ser mais robustas em cenários dinâmicos.